AUTONOMOUS CODE REVIEW
& BUG-FIX AGENT

ML Engineering Project Roadmap

LLM Agents | SWE-bench | DeepSeek-Coder | AST Parsing | RL Fine-Tuning

TARGET BENCHMARKS

30–42%	74%+	< 2.4
SWE-bench Lite Resolved	File Localisation Accuracy	Avg Attempts to Fix

WHY THIS IS HARD — ML DEPTH

- **Context Budget:** Repo has 10 K+ files; context window is ~8 K tokens. Two-stage retrieval + graph propagation solves this.
- **Binary Evaluation:** Tests pass or they don't — no fuzzy gradient. Reward is sparse. Failure categorisation gives signal.
- **Sandboxed Execution:** Agent runs real bash/git/pytest. Network isolation, cgroups, and command whitelists prevent breakage.
- **Self-Correction Loop:** Agent reads its own failing test output and iterates. Trajectories logged for data-driven fine-tuning.
- **Production Economics:** Token cost, retry cost, caching — real systems must be measurable. Cost tracking from day one.

INTERVIEW STORY (20-MIN TALKING POINT)

*"The hardest part was file localisation — a 10 K file repo won't fit in context. I built a two-stage retrieval: BM25 + embeddings for coarse ranking, then a fine-tuned DeBERTa classifier for precision. I also added graph propagation — if a file imports a suspicious module, relevance flows along that edge. That combination lifted localisation recall@5 from **41%** → **74%**, which directly drove pass@1 on SWE-bench. I deliberately fine-tuned after building the reflection loop — so the training data was real observed failures, not guesses."*

1 ENVIRONMENT & BASELINE SETUP

Week 1–2

Goal: Stand up the end-to-end skeleton so every later component has a place to plug in. Scope: Python repos only — JS/Java add parser & sandbox complexity without SWE-bench benefit.

TASKS

- Clone SWE-bench Lite dataset (300 real Python GitHub issues + verified patches).
- Build Docker sandbox: Ubuntu 22.04, git, pytest, conda env per repo — Python-only scope keeps infra clean.
- Sandbox security: network isolation (--network=none), CPU/memory cgroups, command whitelist, 60s timeout — document these for interview.
- Implement naive baseline: feed raw issue text → GPT-4o → apply diff → run tests.
- Measure baseline % resolved (expect ~10–18%) — this is your improvement target.
- Set up experiment tracking with MLflow or W&B; log token cost per issue from day one.

✦ DELIVERABLE: **Reproducible Docker image + baseline numbers on SWE-bench Lite. Security policy doc.**

Docker | Python 3.11 | pytest | SWE-bench | MLflow | cgroups

---

2 AST-AWARE CODE UNDERSTANDING

Week 3–4

Goal: Give the agent structural Python code understanding and a dependency-aware graph — not just raw text.

TASKS

- Integrate Tree-sitter with the Python grammar only — avoids JS/Java parser complexity and conda env conflicts.
- Extract function signatures, class hierarchies, imports, call graphs per file into structured JSON.
- Build a repo dependency graph: nodes = symbols/files, edges = calls/imports (networkx).
- Dependency-aware retrieval scoring: build import/call graph, run Personalized PageRank (networkx ppr) seeded on BM25 top-k files — relevance flows to transitive dependencies. This is the genuinely novel component.
- Cache AST JSON and graph per repo SHA so repeated queries pay zero re-parse cost.
- Unit-test parser on edge cases: decorators, async/await, dataclasses, comprehensions.

✦ DELIVERABLE: **AST parser + dependency graph module; graph-propagation scorer with unit tests. Interview talking point: graph-aware retrieval.**

tree-sitter (Python) | networkx | pygments | ast | diskcache

---

3 TWO-STAGE FILE LOCALISATION + CACHING

Week 5–6

Goal: Find the right 5–10 files from 10 K without exceeding context budget — with graph propagation, caching, and failure categorisation.

TASKS

- Stage 1 — BM25 retrieval: index file paths + docstrings + function names (rank-bm25); hybrid with text-embedding-3-small cosine similarity.

- Hybrid fusion: RRF (Reciprocal Rank Fusion) merges BM25 + embedding + PPR graph-propagation scores; tune alpha weights on validation split.
- Stage 2 — Fine-tune DeBERTa-v3-small classifier on (issue, file_summary) → relevant/not; measure recall@5, recall@10.
- Embedding cache (Redis or diskcache): skip re-embedding files whose content SHA hasn't changed — critical for latency once users arrive.
- Track token cost per retrieval call in MLflow; add per-repo cost estimate to dashboard.
- Categorise localisation failures: wrong-file, partial-file, missing-dependency, ambiguous-issue — log counts per category.

✦ DELIVERABLE: **Localisation pipeline: recall@5 from ~41% → 74%+. Embedding cache. Failure-category breakdown table.**

rank-bm25 | faiss | sentence-transformers | DeBERTa | sklearn | Redis | diskcache

---

4 AGENTIC REFLECTION LOOP

Week 7–8

Goal: Let the agent read its own failing tests and self-correct — up to N attempts. Log every trajectory for data-driven fine-tuning later.

TASKS

- Implement tool-use agent: tools = [read_file, write_patch, run_tests, git_diff]. Celery retry policy: max_retries=3, exponential backoff; sandbox crash → dead-letter queue with structured error.
- After each attempt, parse pytest stdout → extract failure messages → re-prompt with error context.
- Add attempt counter; stop at max_attempts=3 to bound token + compute cost.
- Categorise patch failures per attempt: syntax error, hallucinated API, wrong-file edit, incomplete patch, flaky test — log counts in MLflow.
- Log full trajectories (attempt, patch, test_output, failure_category, success) as JSONL — this becomes the fine-tuning dataset in Phase 7.
- Test reflection loop on 20 hand-picked SWE-bench issues; measure avg attempts to fix.

✦ DELIVERABLE: **Reflection agent achieving avg attempts < 2.4. Trajectory JSONL dataset. Failure-category breakdown.**

LangGraph | tool-use API | subprocess | pytest | json-patch | JSONL

---

5 DEVELOPER PLATFORM UI & DEPLOYMENT

Week 9–10

Goal: Build a live demo UI that visualises the agent's reasoning — because interviewers WILL ask 'Can I see it?'

TASKS

- Frontend (Next.js + Tailwind + Monaco): repo input, issue panel, retrieved-file viewer, patch diff viewer (GitHub-style), test console, reflection logs, confidence gauge.
- Dependency Graph: React Flow visualisation of the repo call-graph with retrieved files highlighted and a retrieval heatmap overlay — the single biggest visual differentiator.
- Streaming Execution Log: WebSocket / SSE feed showing [1/5] Parsing repo... [2/5] Retrieving files... [3/5] Generating patch... [4/5] Running pytest... [5/5] Reflection retry...
- Retry Timeline: visual lane showing Attempt 1 failed → parsed test output → modified reasoning → Attempt 2 success — demonstrates true agentic behaviour.

- Live Metrics Dashboard: issues resolved count, avg runtime, retry rate, localisation accuracy, token cost per issue — makes the system feel operational.
- Backend (FastAPI + WebSockets): /solve endpoint, async task queue (Redis + Celery), retrieval service, execution manager talking to Docker sandbox container.
- Cold-start optimisation: pre-warm AST + embeddings on popular repos at container start; repo fingerprinting (SHA-based) skips re-indexing on identical pushes — p95 latency target < 30s.
- CI/CD: GitHub Actions pipeline — lint (ruff), unit tests (pytest), Docker build check, auto-deploy to Railway on main merge.
- Three-container Docker Compose: frontend (Vercel), API (Railway/Render), sandbox execution container (isolated, --network=none, 60s timeout, memory cap).

✦ DELIVERABLE: **Live public URL — open laptop in interview, paste a GitHub issue, show the agent think and fix in real time.**

Next.js | Tailwind | Monaco Editor | React Flow | FastAPI | WebSockets | Redis+Celery | Docker Compose | Vercel | Railway | GitHub Actions

6 UNCERTAINTY & CONFORMAL PREDICTION

Week 11

Goal: Agent knows when to give up and ask for help — not just guess confidently.

TASKS

- Compute token-level log-probs for generated patch; derive patch confidence score.
- Calibrate using conformal prediction (split-conformal on calibration set).
- If confidence < threshold → emit 'I need human input' instead of bad patch.
- Measure coverage guarantee: true patch in prediction set 90% of the time.
- Pipe confidence score live into the UI confidence dashboard (built in Phase 6).

✦ DELIVERABLE: **Calibrated uncertainty score with provable coverage guarantee, visualised live in demo.**

conformal-prediction | numpy | scipy | matplotlib

7 DATA-DRIVEN FINE-TUNING (DeepSeek-Coder)

Week 12–14

Goal: Fine-tune on real trajectories collected from Phase 4 — optimise known weaknesses, not guesses. Use a 2026-era model, not stale CodeLLaMA.

TASKS

- Select base model: DeepSeek-Coder-7B or Qwen2.5-Coder-7B — both outperform CodeLLaMA-7B on HumanEval/SWE-bench in 2025–26 evals.
- Filter Phase 4 trajectory JSONL: keep only issues where failure category is known (syntax error, wrong-file, hallucinated API) — train on hard cases.
- Format as instruction dataset: system_prompt + issue + localised_file_context + failure_message → corrected_unified_diff.
- Fine-tune with QLoRA (4-bit, r=16, alpha=32) on RunPod A100 (~$40–60 total).
- Evaluate on held-out SWE-bench Lite split: patch validity, test pass rate, token cost vs GPT-4o.
- Ablate: base model vs fine-tuned — report delta in % resolved to justify fine-tuning cost clearly.

✦ DELIVERABLE: **LoRA adapter checkpoint + training curve + ablation table showing improvement over base model.**

DeepSeek-Coder-7B | QLoRA | PEFT | bitsandbytes | transformers | RunPod A100

8 TELEMETRY & PRODUCTION OBSERVABILITY

Week 15

Goal: Instrument real usage — collect failure patterns, latency, and token economics from live traffic. This turns a demo into a system.

TASKS

- Collect usage analytics via PostHog or custom logging: issue type, repo size, latency breakdown by phase.
- Monitor retrieval degradation on large/monorepo inputs — identify where recall@5 drops below 60%.
- Cache warm repos (AST + embeddings) with Redis TTL; measure cache hit rate and p95 latency improvement.
- Track per-issue token cost (retrieval + patch gen + reflection retries); add cost estimate to UI before run.
- Analyse failed trajectory patterns: which failure categories dominate? Feed insights back into training data curation.
- Add rate limiting (slowapi) and request queue depth monitoring to prevent sandbox overload.

✦ **DELIVERABLE: Telemetry dashboard + cost-per-issue report. Interview story: 'After deployment we noticed retrieval degraded on monorepos — here's what we did.'**

PostHog | Redis | Prometheus | Grafana | slowapi | structlog

9 BENCHMARKING & WRITE-UP

Week 16–17

Goal: Produce interview-ready results with measurable, evidence-based claims. No hype — every number is reproducible.

TASKS

- Full SWE-bench Lite eval: % resolved, localisation recall@5, avg attempts, token cost/issue — report all four.
- Ablation table: baseline → +graph retrieval → +caching → +reflection → +fine-tune → +conformal gate.
- Compare vs Devin (13.86%) and SWE-agent (12.47%) — frame as 'on a $100 GPU budget' not 'beats Devin'.
- Failure analysis section: breakdown by category (retrieval miss, syntax error, hallucinated API) — shows research maturity.
- Write 4-page technical report (Overleaf) + GitHub README with architecture diagram + live demo link.
- Record a 3-min demo video using the live UI — agent fixing a real open-source Python bug end-to-end.

✦ **DELIVERABLE: Public GitHub repo, reproducible eval script, paper PDF, live demo URL, demo video, LinkedIn post.**

LaTeX | matplotlib | seaborn | GitHub Actions | Docker Hub

WHY THE UI MATTERS SO MUCH	When an interviewer asks *"Can I see it?"* — you open a live URL, paste a GitHub issue URL, and they watch the agent parse the repo, highlight suspect files on a dependency graph, generate a patch, run pytest, fail, self-correct, and pass. That 90-second demo is worth more than 30 slides. The UI **visualises the intelligence** — retrieval, reasoning, planning, execution, failure correction.

FULL SYSTEM ARCHITECTURE

Component	Technology	Purpose

Sandbox Executor	Docker + cgroups + cmd whitelist	Safe bash/pytest/git; network-isolated
AST Parser	Tree-sitter (Python only)	Structural code + dependency graph
Graph Propagation	networkx + custom scorer	Relevance flows via import/call edges
Stage-1 Retrieval	BM25 + text-embedding-3-small	Coarse ranking; RRF fusion
Stage-2 Ranker	DeBERTa-v3-small (fine-tuned)	Precise file-level localisation
Caching Layer	Redis + diskcache	AST, embedding, test-result caches
Patch Generator	DeepSeek-Coder-7B + QLoRA	Issue → unified diff (2026 SOTA)
Reflection Loop	LangGraph tool-use agent	Self-correction from test failures
Failure Categoriser	Rule-based + regex classifier	Taxonomy: syntax/halluc/wrong-file…
Uncertainty	Conformal Prediction	Coverage-guaranteed confidence gate
Frontend UI	Next.js + Monaco + React Flow	Live demo: graph, diff, logs, metrics
Backend API	FastAPI + WebSockets + Celery	Streaming execution & async queue
Telemetry	PostHog + Prometheus + Grafana	Latency, cost, failure-rate monitoring
Deployment	Vercel / Railway / RunPod	Public live demo; GPU on-demand
Eval Suite	SWE-bench Lite (300 issues)	Reproducible benchmark vs Devin

EXPECTED ABLATION RESULTS

System Variant	SWE-bench % Resolved	Localisation recall@5
Devin (published baseline)	13.86 %	—
SWE-agent (published baseline)	12.47 %	—
Naive GPT-4o baseline	~10–18 %	~41 %
+ Graph-aware two-stage localisation	~25–28 %	~74 %
+ Reflection loop (max 3 tries)	~30–35 %	~74 %
+ Live UI + telemetry (no model change)	~30–35 %	~74 % (visualised)
+ DeepSeek-Coder fine-tune on trajectories	~38–44 %	~74 %
+ Conformal uncertainty gate	**30–42 % (target)**	**~74 %**

KNOWN LIMITATIONS — Engineering Honesty

- **Monorepo retrieval degradation:** Recall@5 drops when repos exceed ~5 K files; graph PPR becomes expensive. Mitigation: shard graph by package boundary.

- **Flaky & non-deterministic tests:** ~8% of SWE-bench issues involve tests that fail/pass randomly. Agent may burn retries on infrastructure noise, not bugs.
- **Python-only scope:** JS, Java, Rust repos are unsupported. SWE-bench is 95% Python, so benchmark numbers don't transfer to polyglot codebases.
- **Cold-start latency:** First request on an uncached repo takes 45–90s (AST + embedding). Subsequent requests hit cache and drop to < 15s.
- **Long-tail patch failures:** Multi-file refactors, API contract changes, and missing test fixtures are reliably hard. Agent escalates via conformal gate.
- **Benchmark ceiling:** Target of 30–42% is realistic; >50% would require larger model + longer context. Claims are framed conservatively on purpose.

Total timeline: ~17 weeks | GPU cost: ~$60–100 (RunPod A100, fine-tuning only) | Target: 30–42% on SWE-bench Lite (300 issues) | Python-only scope | Live demo URL included.